

IF3140 - Sistem Basis Data

Praktikum 0

Review Basis Data Relasional dengan PostgreSQL

14 September 2026



Aurelia Jennifer Gunawan

13524089

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Petunjuk

1. Praktikum ini bertujuan untuk membiasakan Anda dengan lingkungan DBMS PostgreSQL. PostgreSQL akan menjadi DBMS utama dalam rangkaian pembelajaran IF3140 Sistem Basis Data.
2. Setiap peserta kuliah **WAJIB** mengunduh, menginstal, menjalankan, dan mengerjakan praktikum ini pada server PostgreSQL pada perangkat masing masing.
3. Gunakan soal ini sebagai pembiasaan Anda dalam menggunakan PostgreSQL. Risiko tidak mengerjakan praktikum ini ditanggung oleh masing-masing. Soal praktikum akan memiliki tingkat kesulitan yang setara. Pengerjaan dengan batas waktu dapat menjadi opsi untuk berlatih.
4. Seluruh *keyword* yang ada diperbolehkan pada praktikum ini. Silahkan memilih *keyword* yang sesuai dengan kebutuhan soal.

Bioskop

```
film = (idfilm, namafilm, tahun, durasi, imdbrating)
film_has_genre = (idfilm, genre)
film_has_kemampuanfilm = (idfilm, kemampuan)
genre = (genre)
kemampuanfilm = (kemampuan)
kursi = (idkursi, idstudio)
pelanggan = (idpelanggan, namapelanggan)
pelangganonline = (idpelanggan, email, passwordhash)
studio = (idstudio, namastudio)
studio_has_kemampuanfilm = (idstudio, kemampuan)
reservasi = (idfilm, idstudio, waktumulai, idkursi, idpelanggan)
penayangan = (idfilm, idstudio, waktumulai)
penayangan_has_kemampuanfilm = (idfilm, idstudio, waktumulai, kemampuan)
```

Silahkan eksplorasi *primary key*, *foreign key*, dan properti lainnya langsung melalui basis data.

Bagian I. Operasi Dasar PostgreSQL

1. Nyalakan Server PostgreSQL

Jalankan file *docker* yang tersedia.

Command Terminal	<code>docker compose up</code>
SS Terminal dan Hasil	
<pre>PS D:\jen\Kuliah\Praktikum> docker compose up -d [+] up 15/15 ✓Image postgres:16-alpine Pulled ✓Network praktikum_default Created ✓Container praktikum-db-1 Started</pre>	

2. Masuk ke PostgreSQL

Login ke PostgreSQL. Port yang digunakan adalah 5433, dengan **username praktikum** dan **password labdas**

Command Terminal	<code>./psql -h localhost -p 5433 -U praktikum -d bioskop</code>
SS Terminal dan Hasil	
<pre>PS C:\Program Files\PostgreSQL\18\bin> ./psql -h localhost -p 5433 -U praktikum -d bioskop Password for user praktikum: psql (18.6, server 16.15) WARNING: Console code page (437) differs from Windows code page (1252) 8-bit characters might not work correctly. See psql reference page "Notes for Windows users" for details. Type "help" for help. bioskop=#</pre>	

3. Daftar Basis Data

Tampilkan daftar basis data yang ada.

Query	\l																																																						
SS Query dan Hasil																																																							
<pre>biroskop=# \l</pre> <table><thead><tr><th>Name</th><th>Owner</th><th>Encoding</th><th>Locale Provider</th><th>List of databases Collate</th><th>Ctype</th><th>Locale</th><th>ICU Rules</th><th>Access privileges</th></tr></thead><tbody><tr><td>biroskop</td><td>praktikum</td><td>UTF8</td><td>libc</td><td>en_US.utf8</td><td>en_US.utf8</td><td></td><td></td><td></td></tr><tr><td>postgres</td><td>praktikum</td><td>UTF8</td><td>libc</td><td>en_US.utf8</td><td>en_US.utf8</td><td></td><td></td><td></td></tr><tr><td>template0</td><td>praktikum</td><td>UTF8</td><td>libc</td><td>en_US.utf8</td><td>en_US.utf8</td><td></td><td></td><td>=c/praktikum +</td></tr><tr><td>template1</td><td>praktikum</td><td>UTF8</td><td>libc</td><td>en_US.utf8</td><td>en_US.utf8</td><td></td><td></td><td>=c/praktikum +</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>praktikum=CTc/praktikum</td></tr></tbody></table> <pre>(4 rows)</pre>		Name	Owner	Encoding	Locale Provider	List of databases Collate	Ctype	Locale	ICU Rules	Access privileges	biroskop	praktikum	UTF8	libc	en_US.utf8	en_US.utf8				postgres	praktikum	UTF8	libc	en_US.utf8	en_US.utf8				template0	praktikum	UTF8	libc	en_US.utf8	en_US.utf8			=c/praktikum +	template1	praktikum	UTF8	libc	en_US.utf8	en_US.utf8			=c/praktikum +									praktikum=CTc/praktikum
Name	Owner	Encoding	Locale Provider	List of databases Collate	Ctype	Locale	ICU Rules	Access privileges																																															
biroskop	praktikum	UTF8	libc	en_US.utf8	en_US.utf8																																																		
postgres	praktikum	UTF8	libc	en_US.utf8	en_US.utf8																																																		
template0	praktikum	UTF8	libc	en_US.utf8	en_US.utf8			=c/praktikum +																																															
template1	praktikum	UTF8	libc	en_US.utf8	en_US.utf8			=c/praktikum +																																															
								praktikum=CTc/praktikum																																															

4. Menuju Basis Data

Masuk ke basis data bioskop.

Query	\c bioskop
SS Query dan Hasil	
<pre> bioskop=# \c bioskop psql (18.6, server 16.15) You are now connected to database "bioskop" as user "praktikum". </pre>	

5. Daftar Tabel

Tampilkan daftar tabel yang ada pada basis data bioskop.

Query	\dt
SS Query dan Hasil	

```
bioskop=# \dt
```

List of tables			
Schema	Name	Type	Owner
public	_film	table	praktikum
public	_film_has_genre	table	praktikum
public	_film_has_kemampuanfilm	table	praktikum
public	_genre	table	praktikum
public	_kemampuanfilm	table	praktikum
public	_kursi	table	praktikum
public	_pelanggan	table	praktikum
public	_pelangganonline	table	praktikum
public	_penayangan	table	praktikum
public	_penayangan_has_kemampuanfilm	table	praktikum
public	_reservasi	table	praktikum
public	_studio	table	praktikum
public	_studio_has_kemampuanfilm	table	praktikum

(13 rows)

6. Keyword EXPLAIN

Apa itu keyword EXPLAIN?

EXPLAIN adalah keyword yang digunakan untuk menampilkan execution plan, rencana yang dibuat oleh postgresQL untuk menjalankan sebuah query.

Berikan satu contoh penggunaan keyword EXPLAIN. Lalu, jelaskan apa yang terjadi!

Query	EXPLAIN SELECT * FROM _film WHERE tahun > 2000;
SS Query dan Hasil	
<pre>bioskop=# EXPLAIN SELECT * FROM _film WHERE tahun > 2000; QUERY PLAN ----- Seq Scan on _film (cost=0.00..2.56 rows=73 width=28) Filter: (tahun > 2000) (2 rows)</pre>	

Penjelasan
Setelah menjalankan query, postgresSQL menampilkan hasil: QUERY PLAN ----- Seq Scan on _film (cost=0.00..2.56 rows=73 width=28) Filter: (tahun > 2000) (2 rows) yang berarti postgresSQL berencana melakukan sequential scan terhadap tabel _film, yaitu memeriksa baris-baris pada tabel untuk mencari film yang rilis pada tahun setelah 2000.

7. Properti \timing

Apa itu properti \timing?

\timing adalah properti yang digunakan untuk menampilkan waktu yang dibutuhkan postgresSQL untuk menjalankan sebuah query.
--

Berikan satu contoh penggunaan properti \timing. Lalu, jelaskan apa yang terjadi!

Query	\timing on SELECT * FROM _film WHERE tahun > 2014;
SS Query dan Hasil	

```

bioskop=# \timing on
Timing is on.
bioskop=# SELECT * FROM _film WHERE tahun > 2014;
 idfilm |          namafilm          | tahun | durasi | imdbrating
-----+-----+-----+-----+-----
      28 | Jurassic World             |  2015 |    124 |        7.00
     488 | Evolution                  |  2015 |     81 |        6.40
     562 | Minions                    |  2015 |     91 |        6.40
    1209 | Ride Along 2               |  2016 |    102 |        5.90
    1730 | A Warrior's Tail           |  2015 |     85 |        4.10
    1788 | Pride and Prejudice and Zombies |  2016 |    108 |        5.80
    1804 | The Jungle Book            |  2016 |    106 |        7.80
    2019 | Point Break                |  2015 |    114 |        5.30
    3017 | Dancin' It's On           |  2015 |     89 |        2.80
    3130 | Sinister 2                 |  2015 |     97 |        5.20
    3543 | Two Lovers and a Bear      |  2016 |     96 |        7.20
    3933 | Zipper                     |  2015 |    103 |        5.70
(12 rows)

Time: 1,919 ms

```

Penjelasan

Dengan \timing on, postgresQL menampilkan durasi eksekusi setelah query dijalankan. Untuk menonaktifkan properti ini, jalankan \timing off. Properti timing ini berguna untuk membandingkan performa query.

Bagian II. PostgreSQL DDL & DML

Kerjakan No 1-5 terlebih dahulu. Selanjutnya, kerjakan No 6-11 secara berurutan. Untuk seluruh soal, aktifkan \timing dan masukkan hasil perhitungan waktu dalam tangkapan layar.

1. Rekap Penjualan Tiket

Tampilkan berapa jumlah kursi yang dipeservasi per bulan, urutkan dari bulan yang memiliki reservasi kursi paling banyak.

Query	<pre>SELECT SUBSTRING(waktumulai FROM 1 FOR 7) AS bulan, COUNT(*) as jumlah_reservasi FROM _reservasi GROUP BY bulan ORDER BY jumlah_reservasi DESC;</pre>
SS Query dan Hasil	
<pre> bioskop=# SELECT bioskop=# SUBSTRING(waktumulai FROM 1 FOR 7) AS bulan, bioskop=# COUNT(*) as jumlah_reservasi bioskop=# FROM _reservasi bioskop=# GROUP BY bulan bioskop=# ORDER BY jumlah_reservasi DESC; bulan jumlah_reservasi -----+----- 2008-05 2593 2014-01 2583 1900-03 1960 1995-05 1857 2006-01 1813 2001-09 1804 2012-08 1764 1995-04 1717 1995-12 1543 2002-11 1532 1998-08 1416 2011-01 1370 2005-09 1366 1900-12 1333 1993-12 1300 1900-08 1224 1997-06 1188 2004-11 1168 1999-12 1167 2011-12 1153 1998-03 1139 2005-08 1136 2006-06 1135 2015-08 1124 1998-11 1121 1929-07 1116 </pre>	

```

1980-07 | 455
2016-06 | 444
1995-03 | 406
2012-06 | 394
1999-07 | 391
2014-10 | 385
2012-09 | 368
1994-10 | 357
2003-12 | 356
1993-03 | 310
2012-07 | 292
2015-10 | 283
1981-02 | 274
2014-06 | 265
1999-08 | 234
1916-05 | 227
2008-12 | 221
1999-03 | 202
2016-07 | 185
2001-05 | 144
2015-07 | 128
2008-08 | 121
1998-04 | 113
2008-06 | 111
1995-02 | 99
1989-10 | 87
2013-12 | 66
2010-11 | 65
1997-10 | 36
(130 rows)

Time: 46,218 ms
bioskop=#

```

2. Genre Film Terfavorit

Tampilkan genre terfavorit (ditunjukkan dengan penjualan tiket yang paling tinggi) dari seluruh film yang dirilis setelah tahun 1995. Berikan pula total jumlah tiket yang terjual

Query	<pre> SELECT fg.genre, COUNT(*) AS jumlah_tiket_terjual FROM _reservasi r JOIN _film f ON r.idfilm = f.idfilm JOIN _film_has_genre fg ON f.idfilm = fg.idfilm WHERE f.tahun > 1995 GROUP BY fg.genre ORDER BY jumlah_tiket_terjual DESC LIMIT 1; </pre>
SS Query dan Hasil	

```

bioskop=# SELECT fg.genre, COUNT(*) AS jumlah_tiket_terjual
bioskop=# FROM _reservasi r
bioskop=# JOIN _film f ON r.idfilm = f.idfilm
bioskop=# JOIN _film_has_genre fg ON f.idfilm = fg.idfilm
bioskop=# WHERE f.tahun > 1995
bioskop=# GROUP BY fg.genre
bioskop=# ORDER BY jumlah_tiket_terjual DESC
bioskop=# LIMIT 1;
genre | jumlah_tiket_terjual
-----+-----
Drama |                44744
(1 row)

Time: 75,930 ms

```

3. Studio Elite

Studio Elite didefinisikan sebagai studio yang memiliki kemampuan 3D, dan memiliki jumlah kursi lebih dari 150. Tampilkan id, nama, dan jumlah kursi dari seluruh Studio Elite yang ada.

Query	<pre> SELECT s.idstudio, s.namastudio, COUNT(k.idkursi) as jumlah_kursi FROM _studio s JOIN _studio_has_kemampuanfilm shk ON s.idstudio = shk.idstudio JOIN _kursi k ON s.idstudio = k.idstudio WHERE shk.kemampuan = '3D' GROUP BY s.idstudio, s.namastudio HAVING COUNT(k.idkursi) >150 ORDER BY s.idstudio; </pre>
SS Query dan Hasil	

```

bioskop=# SELECT s.idstudio, s.namastudio, COUNT(k.idkursi) as jumlah_kursi
bioskop=# FROM _studio s
bioskop=# JOIN _studio_has_kemampuanfilm shk
bioskop=# ON s.idstudio = shk.idstudio
bioskop=# JOIN _kursi k ON s.idstudio = k.idstudio
bioskop=# WHERE shk.kemampuan = '3D'
bioskop=# GROUP BY s.idstudio, s.namastudio
bioskop=# HAVING COUNT(k.idkursi) >150
bioskop=# ORDER BY s.idstudio;
  idstudio | namastudio | jumlah_kursi
-----+-----+-----
        2 | Studio 3   |          180
        6 | Studio 7   |          180
       10 | Studio 11  |          180
       13 | Studio 14  |          180
       14 | Studio 15  |          200
       16 | Studio 17  |          275
       17 | Studio 18  |          220
(7 rows)

Time: 3,730 ms

```

4. Studio Jauh

Tampilkan id film yang ditayangkan di studio 10 hingga studio 19 pada bulan Januari hingga September tahun 2015 dan memiliki rating lebih dari 6.

Query	<pre> SELECT DISTINCT p.idfilm FROM _penayangan p JOIN _studio s ON p.idstudio = s.idstudio JOIN _film f ON p.idfilm = f.idfilm WHERE p.idstudio BETWEEN 9 AND 18 AND p.waktumulai::timestamp >= '2015-01-01' AND p.waktumulai::timestamp < '2015-10-01' AND f.imdbrating > 6 ORDER BY p.idfilm; </pre>
SS Query dan Hasil	

```

bioskop=# SELECT DISTINCT p.idfilm
bioskop=# FROM _penayangan p
bioskop=# JOIN _studio s ON p.idstudio = s.idstudio
bioskop=# JOIN _film f ON p.idfilm = f.idfilm
bioskop=# WHERE p.idstudio BETWEEN 9 AND 18
bioskop=# AND p.waktumulai::timestamp >= '2015-01-01'
bioskop=# AND p.waktumulai::timestamp < '2015-10-01'
bioskop=# AND f.imdbrating > 6
bioskop=# ORDER BY p.idfilm;
 idfilm
-----
      28
     488
     562
(3 rows)

Time: 2,409 ms

```

5. Komunitas Marcus

Buatlah sebuah view bernama “marcus_movie” yang menampilkan judul film sebagai “Judul”, rating IMDB sebagai “Rating Film”, dan tahun rilis sebagai “Tahun” dari film-film yang pernah dilihat oleh pelanggan yang memiliki nama depan Marcus! Tunjukkan juga bahwa view berhasil dibuat!

Query Jawaban	<pre> CREATE VIEW marcus_movie AS SELECT DISTINCT f.namafilms AS "Judul", f.imdbrating AS "Rating Film", f.tahun AS "Tahun" FROM _film f JOIN _reservasi r ON f.idfilm = r.idfilm JOIN _pelanggan p ON r.idpelanggan = p.idpelanggan WHERE SPLIT_PART(p.namapelanggan, ' ', 1) = 'Marcus'; </pre>
Query Testing	<pre> SELECT * FROM marcus_movie; </pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# SELECT * FROM marcus_movie; ERROR: relation "marcus_movie" does not exist LINE 1: SELECT * FROM marcus_movie; ^ Time: 7,758 ms </pre>	

SS Query Saat Manipulasi Menggunakan Query Jawaban

```

bioskop=# CREATE VIEW marcus_movie AS
bioskop=# SELECT DISTINCT f.namafilms AS "Judul", f.imdbrating AS "Rating Film", f.tahun AS "Tahun"
bioskop=# FROM _film f
bioskop=# JOIN _reservasi r ON f.idfilm = r.idfilm
bioskop=# JOIN _pelanggan p ON r.idpelanggan = p.idpelanggan
bioskop=# WHERE SPLIT_PART(p.namapelanggan, ' ', 1) = 'Marcus';
CREATE VIEW
Time: 55.522 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT * FROM marcus_movie;
      Judul      | Rating Film | Tahun
-----+-----+-----
 40 Days and 40 Nights |         5.60 | 2002
      Formula 51      |         6.30 | 2001
      Niagara         |         7.10 | 1953
Scream: The TV Series |         7.30 | 1900
      Swing Vote      |         6.10 | 2008
(5 rows)

Time: 43.402 ms

```

6. Populer di Genrenya

Satu film dapat memiliki lebih dari satu genre. Buatlah View dengan nama 'above_avg_genre_penayangan' yang menampilkan genre sebagai 'nama_genre' dan jumlah penayangannya sebagai 'jumlah penayangan' untuk genre film yang memiliki jumlah penayangan di atas rata-rata. Tampilkan hasil akhir viewnya!

Query Jawaban

```

CREATE VIEW above_avg_genre_penayangan AS
WITH jumlah_genre AS (
SELECT fg.genre, COUNT(*) AS jumlah_penayangan
FROM _penayangan p
JOIN _film_has_genre fg ON p.idfilm = fg.idfilm
GROUP BY fg.genre
), rata_rata AS (
SELECT AVG(jumlah_penayangan) AS avg_penayangan
FROM jumlah_genre
)

```

	<pre>SELECT jg.genre AS nama_genre, jg.jumlah_penayangan AS "jumlah penayangan" FROM jumlah_genre jg CROSS JOIN rata_rata r WHERE jg.jumlah_penayangan > r.avg_penayangan;</pre>
Query Testing	<pre>SELECT * FROM above_avg_genre_penayangan ORDER BY "jumlah penayangan" DESC;</pre>

SS Query Sebelum Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT * FROM above_avg_genre_penayangan
bioskop=# ORDER BY "jumlah penayangan" DESC;
ERROR:  relation "above_avg_genre_penayangan" does not exist
LINE 1: SELECT * FROM above_avg_genre_penayangan
                        ^
Time: 10,309 ms

```

SS Query Saat Manipulasi Menggunakan Query Jawaban

```

bioskop=# CREATE VIEW above_avg_genre_penayangan AS
bioskop=# WITH jumlah_genre AS(
bioskop(# SELECT fg.genre, COUNT(*) AS jumlah_penayangan
bioskop(# FROM _penayangan p
bioskop(# JOIN _film_has_genre fg ON p.idfilm = fg.idfilm
bioskop(# GROUP BY fg.genre
bioskop(# ), rata_rata AS (
bioskop(# SELECT AVG(jumlah_penayangan) AS avg_penayangan
bioskop(# FROM jumlah_genre
bioskop(# )
bioskop=#
bioskop=# SELECT jg.genre AS nama_genre, jg.jumlah_penayangan AS "jumlah penayangan"
bioskop=# FROM jumlah_genre jg
bioskop=# CROSS JOIN rata_rata r
bioskop=# WHERE jg.jumlah_penayangan > r.avg_penayangan;
CREATE VIEW
Time: 20,230 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT * FROM above_avg_genre_penayangan
bioskop=# ORDER BY "jumlah penayangan" DESC;
nama_genre | jumlah_penayangan
-----+-----
Drama      |          592
Thriller   |          294
Crime      |          286
Comedy     |          240
Romance    |          191
Action     |          171
Adventure  |          162
Mystery    |          136
(8 rows)

Time: 11,402 ms
bioskop=#

```

7. Tahun Baru Bersama Pacar

Veronica Doheny selalu membuat sebuah reservasi dengan pacarnya dengan menonton film yang tayang di tahun baru. Tetapi, mereka tidak akan menonton film yang memiliki rating dibawah 7. Namun, pada pada malam tahun baru, selalu saja terdapat bug sehingga transaksi yang ia lakukan tidak tercatat. Bantu pegawai untuk menambahkan pemesanan tiket pada tahun baru 2011. (Kerjakan hanya dengan 1 query saja, untuk letak kursi dibebaskan asalkan bersebelahan dengan asumsi seluruh kursi masih kosong)

Query Jawaban	<pre> INSERT INTO _reservasi(idfilm, idstudio, waktumulai, idkursi, idpelanggan) SELECT pen.idfilm, pen.idstudio, pen.waktumulai, k.idkursi, pel.idpelanggan FROM _penayangan pen JOIN _film f ON f.idfilm = pen.idfilm JOIN _kursi k ON k.idstudio = pen.idstudio JOIN _pelanggan pel ON pel.namapelanggan = 'Veronica Doheny' WHERE pen.waktumulai LIKE '2011-01-01%' AND f.imdbrating >= 7 AND k.idkursi IN ('A1','A2');</pre>
Query Testing	<pre> SELECT * FROM _reservasi WHERE idpelanggan = 15932 AND waktumulai LIKE '2011-01-01%';</pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# SELECT * FROM _reservasi WHERE idpelanggan = 15932 AND waktumulai LIKE '2011-01-01%'; idfilm idstudio waktumulai idkursi idpelanggan -----+-----+-----+-----+----- (0 rows) Time: 57,182 ms</pre>	
SS Query Saat Manipulasi Menggunakan Query Jawaban	


```

bioskop=# INSERT INTO _reservasi(idfilm, idstudio, waktumulai, idkursi, idpelanggan)
bioskop=# SELECT pen.idfilm, pen.idstudio, pen.waktumulai, k.idkursi, pel.idpelanggan
bioskop=# FROM _penayangan pen
bioskop=# JOIN _film f ON f.idfilm = pen.idfilm
bioskop=# JOIN _kursi k ON k.idstudio = pen.idstudio
bioskop=# JOIN _pelanggan pel ON pel.namapelanggan = 'Veronica Doheny'
bioskop=# WHERE pen.waktumulai LIKE '2011-01-01%'
bioskop=# AND f.imdbrating >= 7
bioskop=# AND k.idkursi IN ('A1','A2');
INSERT 0 2
Time: 56,558 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT * FROM _reservasi WHERE idpelanggan = 15932 AND waktumulai LIKE '2011-01-01%';
 idfilm | idstudio | waktumulai | idkursi | idpelanggan
-----+-----+-----+-----+-----
    2420 |        3 | 2011-01-01 17:00:00 | A1 | 15932
    2420 |        3 | 2011-01-01 17:00:00 | A2 | 15932
(2 rows)
Time: 15,569 ms

```

8. Rating Masking

Pihak bioskop ingin memperbarui rating beberapa film berdasarkan jumlah penontonnya. Buatlah query untuk mengupdate rating film yang memiliki jumlah penonton diatas 500 dan rating lebih kecil dari 6.00 menjadi 6.00. Tunjukkan bahwa tidak ada lagi film yang memiliki penonton diatas 500 yang memiliki rating dibawah 6.00.

Query Jawaban	<pre> UPDATE _film SET imdbrating = 6.00 WHERE imdbrating < 6.00 AND idfilm IN (SELECT idfilm FROM _reservasi GROUP BY idfilm HAVING COUNT(*) > 500); </pre>
Query Testing	<pre> SELECT f.idfilm, f.namafilam, f.imdbrating, COUNT(r.*) AS jumlah_penonton FROM _film f JOIN _reservasi r ON f.idfilm = r.idfilm </pre>

```
GROUP BY f.idfilm, f.namafilm, f.imdbrating
HAVING COUNT(r.*) > 500 AND f.imdbrating < 6.00;
```

SS Query Sebelum Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT f.idfilm, f.namafilm, f.imdbrating, COUNT(r.*) AS jumlah_penonton
bioskop=# FROM _film f
bioskop=# JOIN _reservasi r ON f.idfilm = r.idfilm
bioskop=# GROUP BY f.idfilm, f.namafilm, f.imdbrating
bioskop=# HAVING COUNT(r.*) > 500 AND f.imdbrating < 6.00;
 idfilm |          namafilm          | imdbrating | jumlah_penonton
-----+-----+-----+-----
  3218 | The Brown Bunny           | 5.00      | 556
  4977 | The FP                    | 5.60      | 665
  3912 | The Night Listener        | 5.90      | 631
  1788 | Pride and Prejudice and Zombies | 5.80      | 662
   505 | Pompeii                   | 5.60      | 559
  3394 | Woman on Top              | 5.30      | 501
  3294 | Breakin' All the Rules    | 5.50      | 609
   564 | Snake Eyes                | 5.90      | 692
   499 | Babe: Pig in the City     | 5.80      | 661
  1690 | The Whole Ten Yards       | 5.50      | 819
  1334 | Against the Ropes         | 5.30      | 518
   758 | Under Siege 2: Dark Territory | 5.40      | 505
  2019 | Point Break               | 5.30      | 515
  3945 | Among Giants              | 5.90      | 597
  3933 | Zipper                    | 5.70      | 618
  2855 | No Good Deed              | 5.60      | 582
  2497 | 40 Days and 40 Nights     | 5.60      | 621
   343 | Jurassic Park III         | 5.90      | 694
  1209 | Ride Along 2              | 5.90      | 629
(19 rows)

Time: 51,353 ms

```

SS Query Saat Manipulasi Menggunakan Query Jawaban

```

bioskop=# UPDATE _film
bioskop=# SET imdbrating = 6.00
bioskop=# WHERE imdbrating < 6.00
bioskop=# AND idfilm IN (
bioskop(#   SELECT idfilm FROM _reservasi
bioskop(#   GROUP BY idfilm
bioskop(#   HAVING COUNT(*) > 500
bioskop(# );
UPDATE 19
Time: 30,511 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT f.idfilm, f.namafil, f.imdbrating, COUNT(r.*) AS jumlah_penonton
bioskop=# FROM _film f
bioskop=# JOIN _reservasi r ON f.idfilm = r.idfilm
bioskop=# GROUP BY f.idfilm, f.namafil, f.imdbrating
bioskop=# HAVING COUNT(r.*) > 500 AND f.imdbrating < 6.00;
 idfilm | namafil | imdbrating | jumlah_penonton
-----+-----+-----+-----
(0 rows)

Time: 44,328 ms

```

9. SPG Studio

Buatlah sebuah tabel dengan nama SPG yang berisi id karyawan, id studio, id film yang sedang ditayangkan. Perhatikan kolom yang harus direferensikan ke table lain.

Query Jawaban	<pre> CREATE TABLE SPG(idkaryawan INTEGER NOT NULL, idstudio SMALLINT NOT NULL, idfilm SMALLINT NOT NULL, PRIMARY KEY (idkaryawan, idstudio, idfilm), FOREIGN KEY (idstudio) REFERENCES _studio(idstudio), FOREIGN KEY (idfilm) REFERENCES _film(idfilm)); </pre>
Query Testing	<pre> INSERT INTO SPG VALUES(1,0,999999); INSERT INTO SPG VALUES(1, 0, 28); SELECT * FROM SPG; </pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# SELECT * FROM SPG; ERROR: relation "spg" does not exist LINE 1: SELECT * FROM SPG; ^ Time: 6,894 ms </pre>	
SS Query Saat Manipulasi Menggunakan Query Jawaban	

```

bioskop=# CREATE TABLE SPG(
bioskop(# idkaryawan INTEGER NOT NULL,
bioskop(# idstudio SMALLINT NOT NULL,
bioskop(# idfilm SMALLINT NOT NULL,
bioskop(# PRIMARY KEY (idkaryawan, idstudio, idfilm),
bioskop(# FOREIGN KEY (idstudio) REFERENCES _studio(idstudio),
bioskop(# FOREIGN KEY (idfilm) REFERENCES _film(idfilm)
bioskop(# );
CREATE TABLE
Time: 31,792 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT * FROM SPG;
 idkaryawan | idstudio | idfilm
-----+-----+-----
(0 rows)

Time: 4,276 ms
bioskop=# INSERT INTO SPG VALUES(1,0,999999);
ERROR: smallint out of range
Time: 9,162 ms
bioskop=# INSERT INTO SPG VALUES(1, 0, 28);
INSERT 0 1
Time: 49,620 ms
bioskop=# SELECT * FROM SPG;
 idkaryawan | idstudio | idfilm
-----+-----+-----
           1 |         0 |      28
(1 row)

Time: 2,293 ms

```

10. Tally Counter

Setiap kali ada pelanggan melakukan reservasi, maka jumlah penonton film tersebut harus otomatis bertambah. Untuk itu:

1. Tambahkan kolom baru jumlahpenonton pada tabel film dengan nilai awal 0.
2. Buat trigger pada tabel reservasi yang akan menambahkan +1 pada kolom jumlah penonton di tabel film sesuai idfilm yang dipesan.

Query Case	<pre>SELECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpenonton FROM _film ORDER BY jumlahpenonton DESC;</pre>
SS Query Case dan Hasil	
<pre>bioskop=# SELECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpenonton bioskop=# FROM _film bioskop=# ORDER BY jumlahpenonton DESC; ERROR: column "jumlahpenonton" does not exist LINE 1: ...LECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpeno... ^ Time: 9,883 ms</pre>	
Query Jawaban	<pre>ALTER TABLE _film ADD COLUMN jumlahpenonton INTEGER DEFAULT 0; CREATE OR REPLACE FUNCTION fn_tally_counter() RETURNS TRIGGER AS \$\$ BEGIN UPDATE _film SET jumlahpenonton = jumlahpenonton + 1 WHERE idfilm = NEW.idfilm; RETURN NEW; END; \$\$ LANGUAGE plpgsql; CREATE TRIGGER trg_tally_counter AFTER INSERT ON _reservasi FOR EACH ROW EXECUTE FUNCTION fn_tally_counter();</pre>
SS Query Jawaban dan Hasil	

```

bioskop=# ALTER TABLE _film ADD COLUMN jumlahpenonton INTEGER DEFAULT 0;
ALTER TABLE
Time: 10,899 ms
bioskop=#
bioskop=# CREATE OR REPLACE FUNCTION fn_tally_counter()
bioskop=# RETURNS TRIGGER AS $$
bioskop$$ BEGIN
bioskop$$ UPDATE _film
bioskop$$ SET jumlahpenonton = jumlahpenonton + 1
bioskop$$ WHERE idfilm = NEW.idfilm;
bioskop$$ RETURN NEW;
bioskop$$ END;
bioskop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
Time: 26,439 ms
bioskop=#
bioskop=# CREATE TRIGGER trg_tally_counter
bioskop=# AFTER INSERT ON _reservasi
bioskop=# FOR EACH ROW
bioskop=# EXECUTE FUNCTION fn_tally_counter();
CREATE TRIGGER
Time: 6,082 ms

```

Query
Testing

```

INSERT INTO _reservasi (idfilm, idstudio, waktumulai,
idkursi, idpelanggan)
VALUES (1, 1, '2025-10-01 19:00:00', 101, 1001);

```

SS Query Case dan Hasil Setelah Query Testing

```

bioskop=# INSERT INTO _reservasi (idfilm, idstudio, waktumulai, idkursi, idpelanggan)
bioskop=# VALUES (1, 1, '2025-10-01 19:00:00', 101, 1001);
INSERT 0 1
Time: 9,696 ms

```

11. Autofill

Buatlah trigger agar setiap kali ada data baru masuk ke tabel reservasi tanpa diisi kolom waktumulai, sistem akan secara otomatis mengisi kolom waktumulai sesuai jadwal dari tabel penayangan (berdasarkan idfilm dan idstudio).

Query
Case

```
SELECT * FROM _reservasi;
```

SS Query Case dan Hasil

```

bioskop=# SELECT * FROM _reservasi;
idfilm | idstudio |      waktumulai      | idkursi | idpelanggan
-----+-----+-----+-----+-----
  28 |      0 | 2015-09-30 09:00:00 | A10     |      14110
  28 |      0 | 2015-09-30 09:00:00 | A11     |      14110
  28 |      0 | 2015-09-30 09:00:00 | A12     |      14110
  28 |      0 | 2015-09-30 09:00:00 | A13     |      14110
  28 |      0 | 2015-09-30 09:00:00 | A7      |     10717
  28 |      0 | 2015-09-30 09:00:00 | A8      |     10717
  28 |      0 | 2015-09-30 09:00:00 | A9      |      14110
  28 |      0 | 2015-09-30 09:00:00 | B1      |     17509
  28 |      0 | 2015-09-30 09:00:00 | B10     |     10113
  28 |      0 | 2015-09-30 09:00:00 | B11     |     10113
  28 |      0 | 2015-09-30 09:00:00 | B12     |     10113
  28 |      0 | 2015-09-30 09:00:00 | B2      |     17509
  28 |      0 | 2015-09-30 09:00:00 | B3      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B4      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B5      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B6      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B7      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B8      |     10113
  28 |      0 | 2015-09-30 09:00:00 | B9      |     10113
  28 |      0 | 2015-09-30 09:00:00 | C1      |     17509
  28 |      0 | 2015-09-30 09:00:00 | C14     |      4258
  28 |      0 | 2015-09-30 09:00:00 | C15     |      4258
  28 |      0 | 2015-09-30 09:00:00 | C2      |      5975
  28 |      0 | 2015-09-30 09:00:00 | C3      |      5975
  28 |      0 | 2015-09-30 09:00:00 | C4      |      5975
  28 |      0 | 2015-09-30 09:00:00 | C5      |      5975
  28 |      0 | 2015-09-30 09:00:00 | C6      |      5975
  28 |      0 | 2015-09-30 09:00:00 | C7      |      5975

```

Query
Jawaban

```

CREATE OR REPLACE FUNCTION fn_autofill_waktumulai()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.waktumulai IS NULL THEN
        SELECT waktumulai INTO NEW.waktumulai
        FROM _penayangan
        WHERE idfilm = NEW.idfilm AND idstudio = NEW.idstudio
        LIMIT 1;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

	<pre>CREATE TRIGGER trg_autofill_waktumulai BEFORE INSERT ON _reservasi FOR EACH ROW EXECUTE FUNCTION fn_autofill_waktumulai();</pre>
SS Query Jawaban dan Hasil	
	<pre> bioskop=# CREATE OR REPLACE FUNCTION fn_autofill_waktumulai() bioskop=# RETURNS TRIGGER AS \$\$ bioskop\$# BEGIN bioskop\$# IF NEW.waktumulai IS NULL THEN bioskop\$# SELECT waktumulai INTO NEW.waktumulai bioskop\$# FROM _penayangan bioskop\$# WHERE idfilm = NEW.idfilm AND idstudio = NEW.idstudio bioskop\$# LIMIT 1; bioskop\$# END IF; bioskop\$# RETURN NEW; bioskop\$# END; bioskop\$# \$\$ LANGUAGE plpgsql; CREATE FUNCTION Time: 46,807 ms bioskop=# CREATE TRIGGER trg_autofill_waktumulai bioskop=# BEFORE INSERT ON _reservasi bioskop=# FOR EACH ROW bioskop=# EXECUTE FUNCTION fn_autofill_waktumulai(); CREATE TRIGGER Time: 4,585 ms </pre>
Query Testing	<pre> INSERT INTO _reservasi (idfilm, idstudio, idkursi, idpelanggan) VALUES (28, 0, 'A2', 2001); SELECT * FROM _reservasi WHERE idfilm = 28 AND idstudio = 0 AND idkursi = 'A2' AND idpelanggan=2001; (id dapat diubah sesuai kebutuhan) </pre>
SS Query Case dan Hasil Setelah Query Testing	


```
bioskop=# INSERT INTO _reservasi (idfilm, idstudio, idkursi, idpelanggan)
bioskop=# VALUES (28, 0, 'A2', 2001);
INSERT 0 1
Time: 5,888 ms
bioskop=#
bioskop=# SELECT * FROM _reservasi
bioskop=# WHERE idfilm = 28 AND idstudio = 0 AND idkursi = 'A2' AND idpelanggan=2001;
 idfilm | idstudio |      waktumulai      | idkursi | idpelanggan
-----+-----+-----+-----+-----
      28 |         0 | 2015-09-30 09:00:00 | A2      |          2001
(1 row)

Time: 14,734 ms
```